



Ohanafy Native

One native iOS app for every front-line role — sales reps, warehouse teams, and delivery drivers. Proof of concept, running today: all three journeys, fully offline, with Salesforce as the system of record.

Offline-first

Every workflow completes with zero signal — orders, surveys, payments queue and sync themselves.

Salesforce stays the truth

Every change is validated by Salesforce business logic. The phone proposes; the platform decides.

Beyond the app icon

Lock screen, Dynamic Island, home-screen widget, geofenced arrival, Siri — surfaces the Salesforce mobile app cannot reach.

INTERNAL · STAKEHOLDER REVIEW

September 2026 · Proof of concept · Sample data throughout

Ohanafy Engineering

WHAT THIS IS

A working app, not a slideware concept

Every screenshot in this document was captured from a real SwiftUI app running in the iOS Simulator. It implements all three front-line journeys from the Front-Line Journey mockups — the sales rep's day, the warehouse shift, and the driver's route — as switchable modules over one shared offline core. In production, login decides which journey a user lands in; a switcher behind the avatar covers multi-role users.

How offline works

Every rep action becomes a command in a durable on-device queue. Commands replay to the server in order when connectivity returns — no "save failed," no lost work, no separate offline mode.

How Salesforce stays in charge

Each command carries a unique ID that Salesforce treats as an idempotent upsert: replays can never duplicate an order. Business rules run server-side — a credit hold rejects the order and tells the rep why.

What the rep feels

Screens read only from the on-device store, so everything is instant. Offline and online are the same experience; the only difference is a status pill.

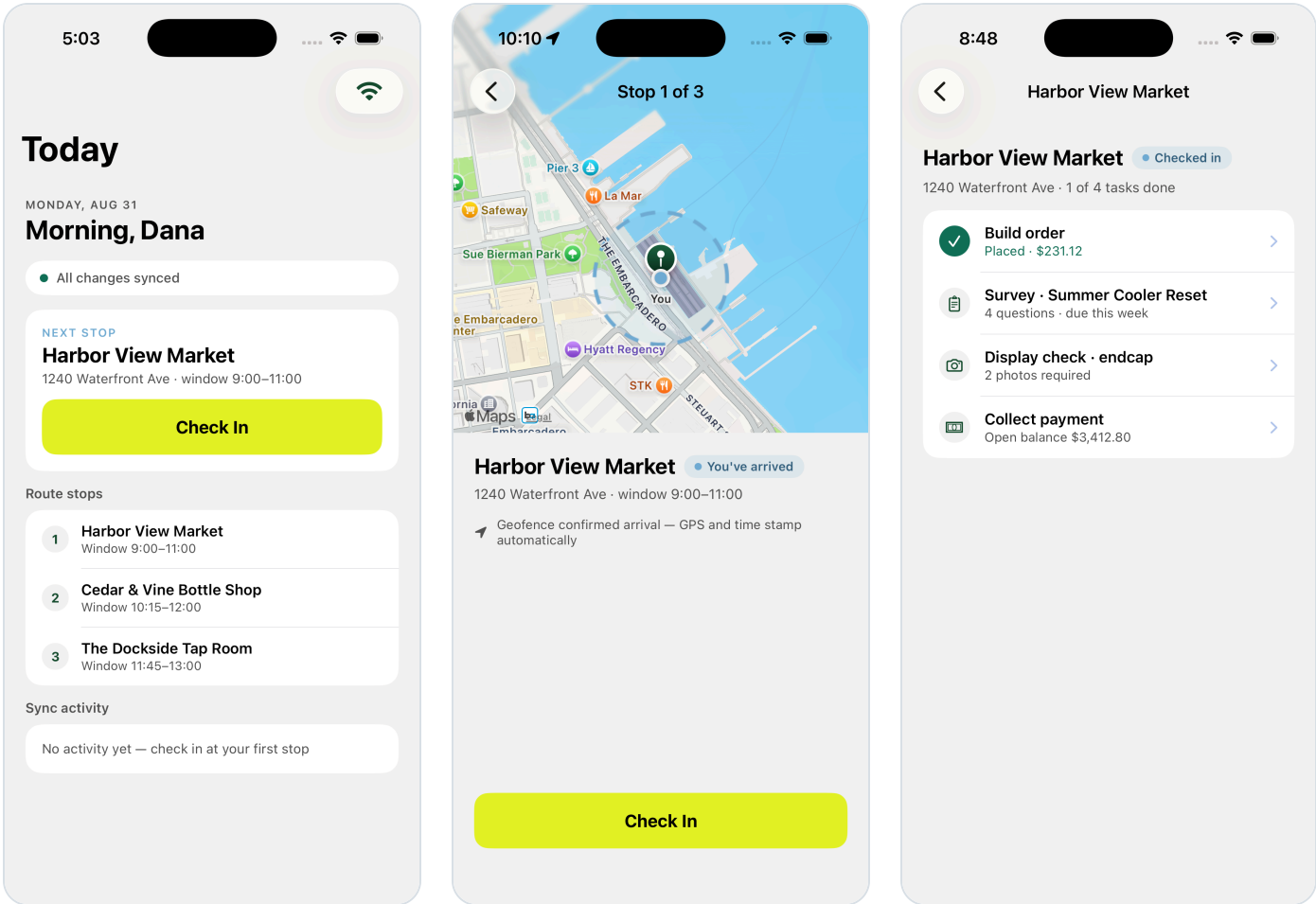
PROOF UNDER THE UI

The sync core is a standalone Swift library with an automated scenario suite. It proves, in executable tests rather than promises: queued work survives app kills and device restarts; commands drain strictly in the order the rep acted; a replayed command applies exactly once; a rejected order never blocks the check-ins and payments queued behind it; and reconnecting after hours offline requires no user action.

Honest scope: this POC uses invented sample data and a mock server that plays Salesforce's role, including its rejections and approval holds. Real Salesforce REST integration, delta sync, authentication, and photo upload are the next milestone (Track A below) — the seams for them are already in the code.

Start the day, arrive, open the stop

The morning route is on the phone before the rep leaves the warehouse. Arrival at each stop is detected, not typed.



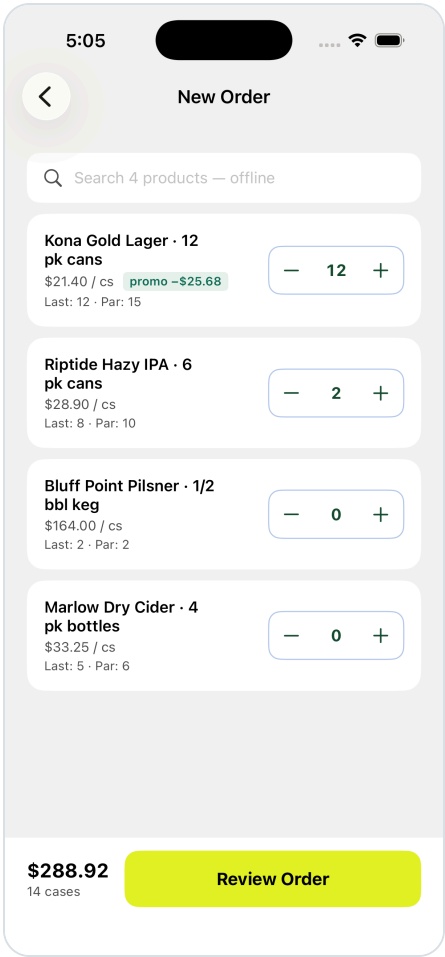
1 Today
The route, next stop, and sync state — rendered entirely from the on-device store. Loads instantly in airplane mode.

2 Arrive — geofence confirmed
A 150-meter geofence around the next stop detects arrival automatically. Check-in stamps real GPS and time with one tap.

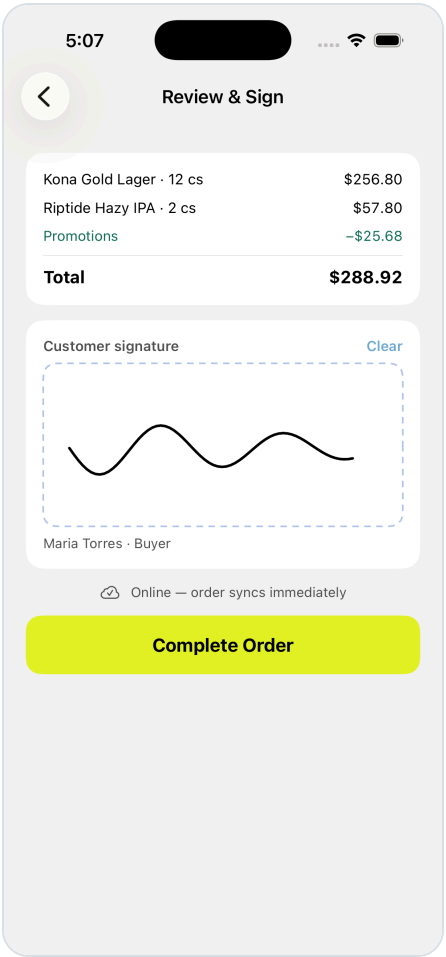
3 The stop hub
Everything due at this account in one checklist: order, survey, display check, payment. Progress tracks as tasks complete.

Sell — with pricing that never waits on signal

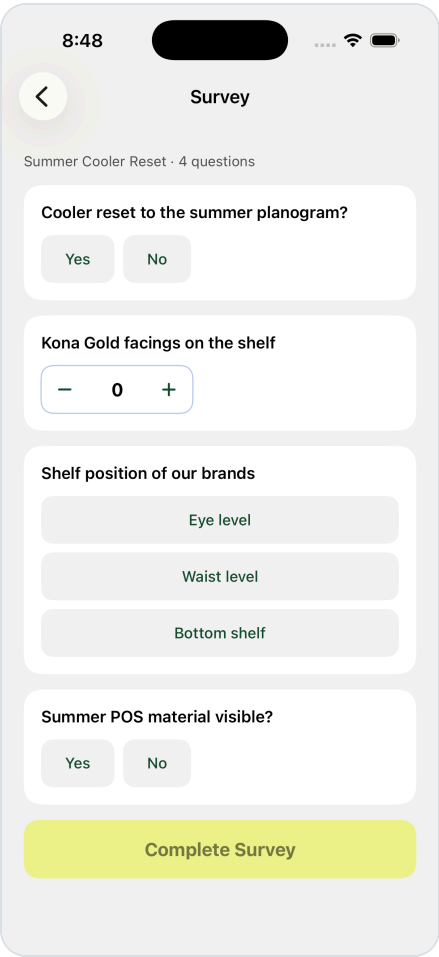
The catalog, promotion rules, and pricing math live on the device. Building an order in a signal-dead cooler aisle feels identical to building one on Wi-Fi.



4 Order capture
Catalog search and promotion pricing run entirely on-device — the 10-case volume discount appears live as quantities change.



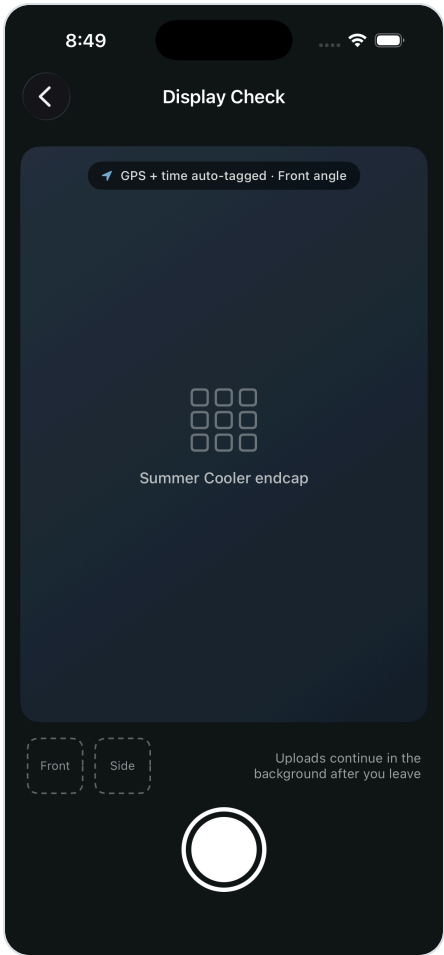
5 Review & sign
Totals with promotion savings and a finger-drawn signature. Completing queues the order — connected or not.



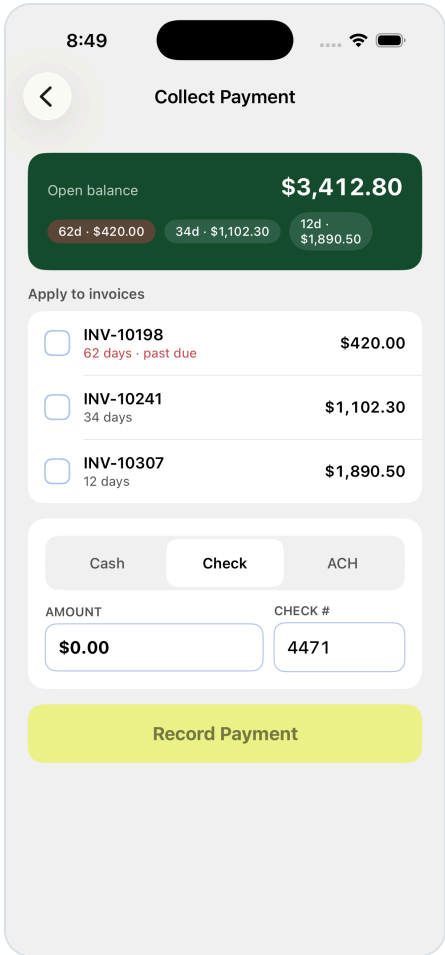
6 In-store survey
The Summer Cooler Reset survey: yes/no, counts, and choice questions, completed at the shelf with no signal required.

Execute and collect

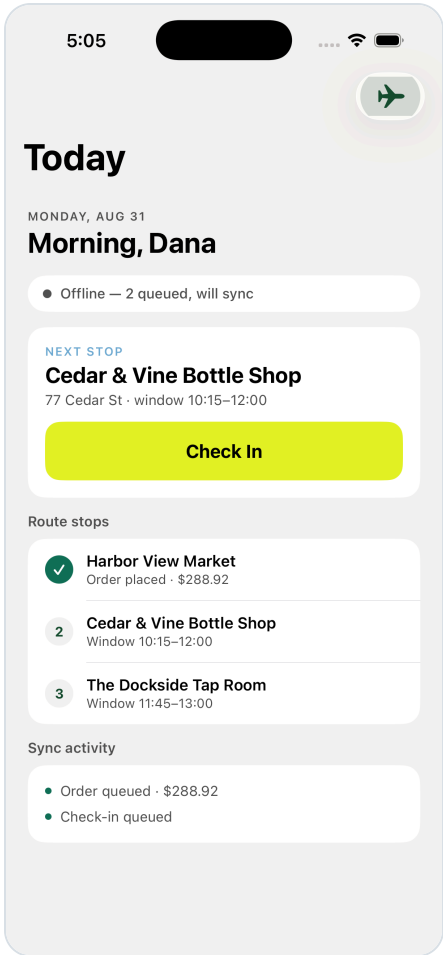
Retail execution and collections ride the same offline queue as orders — every task at the stop completes without a connection.



7 Display check
Camera capture with GPS and time auto-tagged. In production, photos upload in the background — even after the rep pockets the phone.



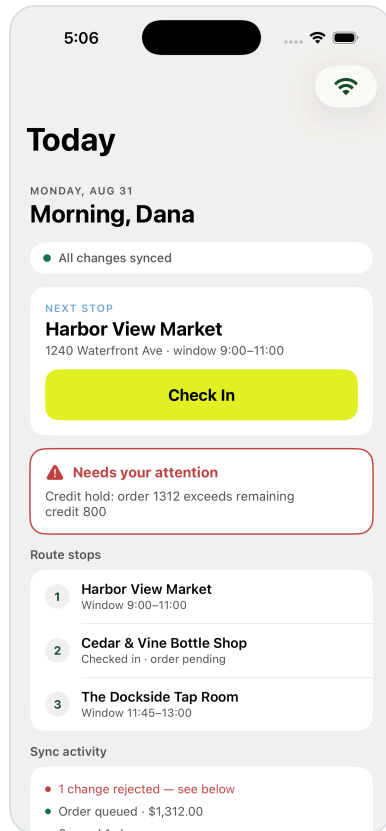
8 Collect payment
Open invoices with aging (past-due flagged), applied by selection, with cash/check/ACH capture. Records offline like everything else.



9 Offline, visibly safe
Airplane mode on: the check-in and a \$288.92 order sit queued. The stop already shows the order placed — the rep keeps moving.

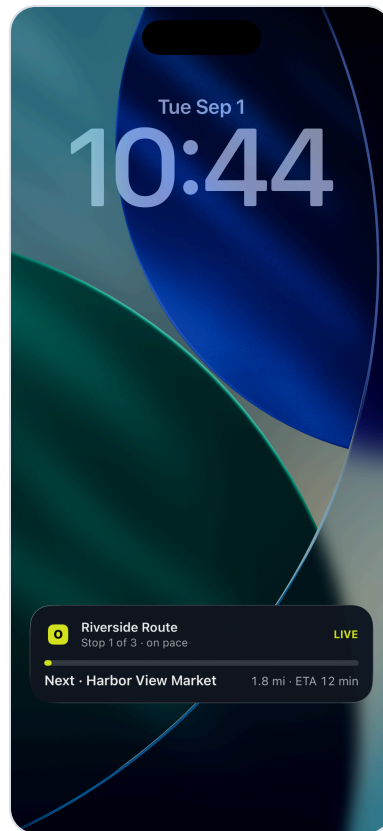
Sync that answers back, presence beyond the app

Salesforce remains the authority: it can refuse a change, and the app makes that a moment the rep can act on rather than a silent failure. Between stops, the route lives on the lock screen.



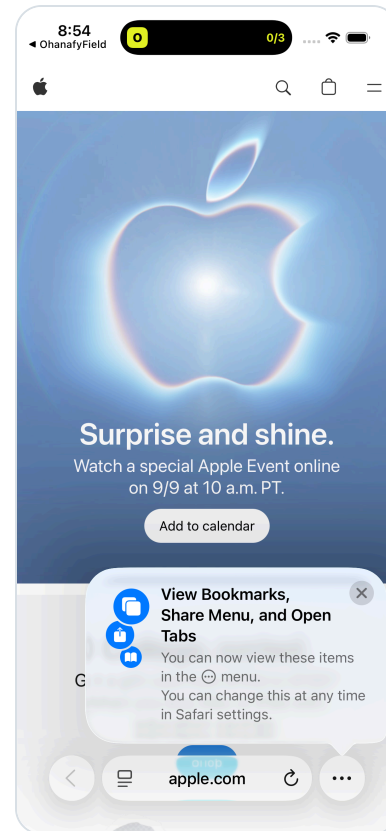
10 Salesforce says no — gracefully

A \$1,312 order against an \$800 credit limit syncs, is refused by the credit hold, and surfaces with the reason. Nothing queued behind it was blocked.



11 The route lives on the lock screen

The Live Activity between stops: route, progress, next stop with distance and ETA — zero app opens. Updates with every action, and by push in production.



12 ...and in the Dynamic Island

The same card, compact, while a different app is on screen — brand mark and route progress always one glance away.

"Hey Siri, what's my next stop?"

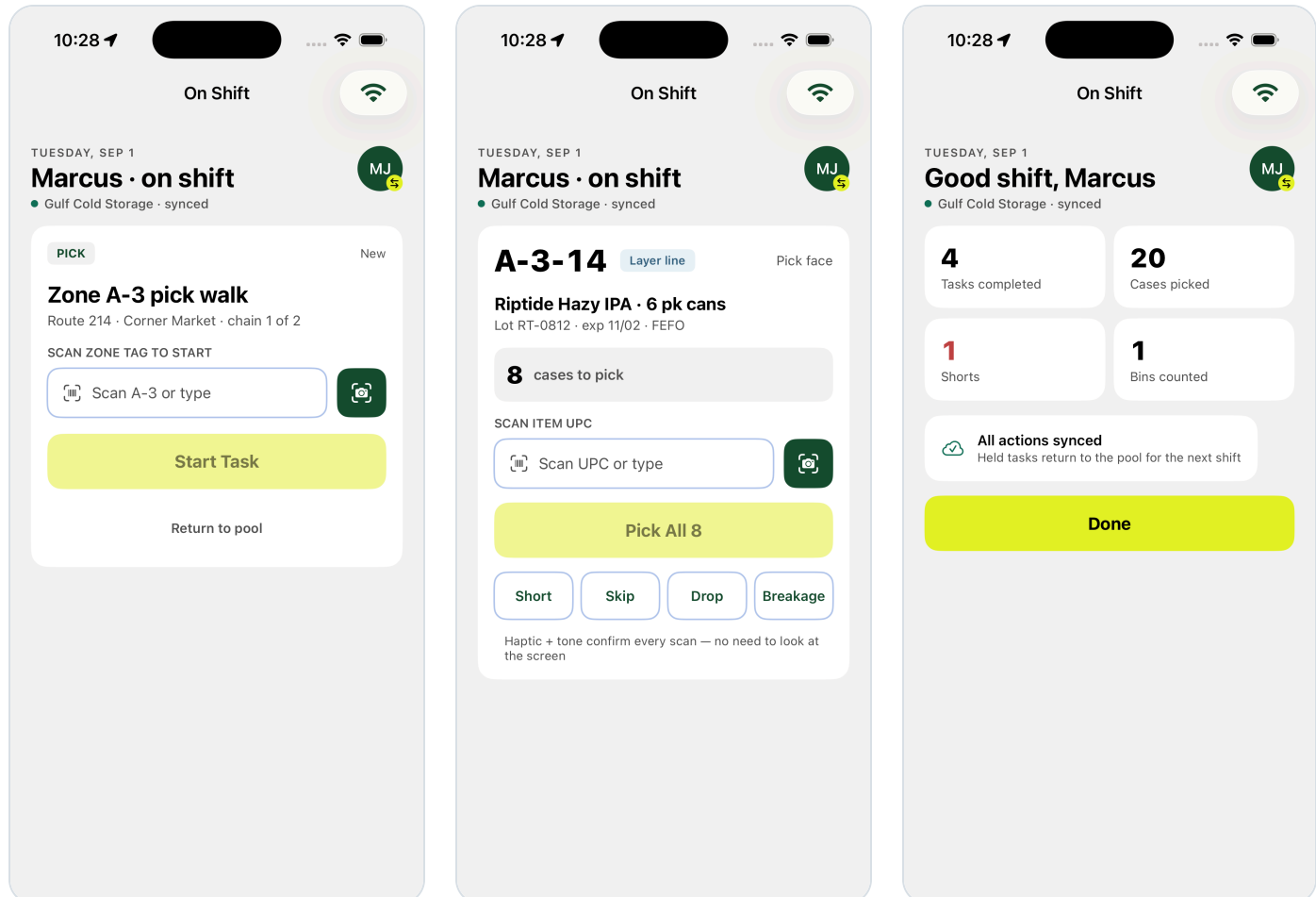
Real Siri / Shortcuts integration in this build: the answer — next stop, window, progress — comes from on-device data, hands-free, without opening the app. Screenshot on the iOS in Action page.

Route on the home screen

A WidgetKit widget shows route progress, next stop, and sync state at a glance; it updates with every action in the app.

Second journey: one task in hand, never a queue

The warehouse module follows the shipped work-task engine's rules: the worker sees exactly one task, every scan input is camera + keyboard-wedge, and completing a task fetches the next in the same gesture. Switch journeys from the avatar, top right.



1 The offer

Start Day pulls one task from the pool — a pick walk, chained by zone. A scan gate on the zone tag proves the worker is standing in the right place before Start unlocks.

2 The pick stop

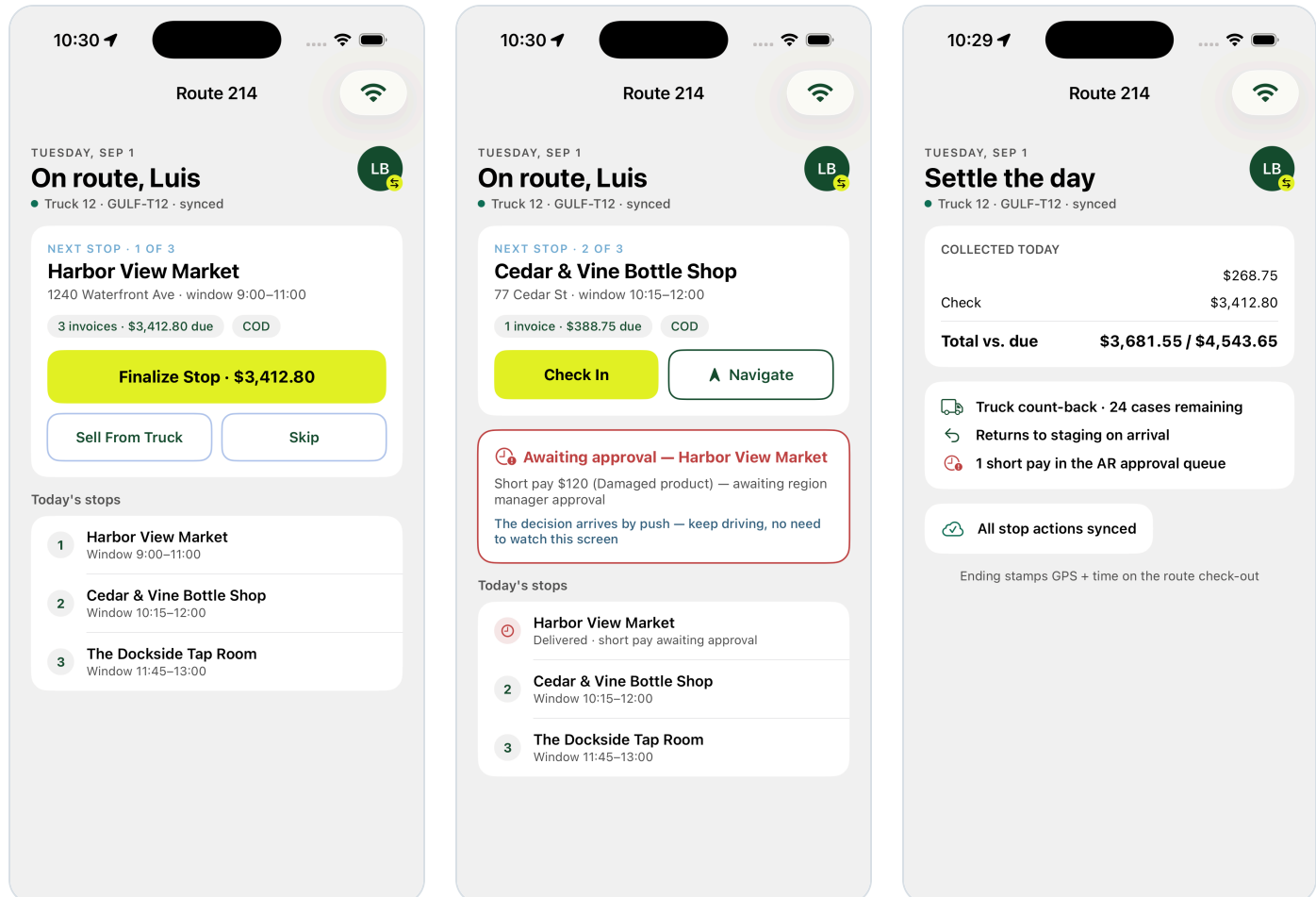
Bin, item, lot, and case count with a UPC scan gate — plus the four shipped exception paths: Short (with reason), Skip, Drop, Breakage.

3 Shift summary

End Day tallies tasks, cases, shorts, and counted bins; held work returns to the pool for the next shift. Put-away scan legs and blind cycle counts run the same in-task pattern.

Third journey: deliver, collect, and let approvals catch up

The driver module mirrors the shipped delivery flow — Start Route gated on the truck being loaded, per-stop finalize with payment capture, sell-from-truck against the truck's own inventory — and turns the short-pay approval wait into something the driver can drive away from.



1 On route

Next stop with amount due, one-tap Check In, and Navigate handing coordinates to Apple Maps. Skip requires a reason, exactly like the shipped flow.

2 Short-pay hold

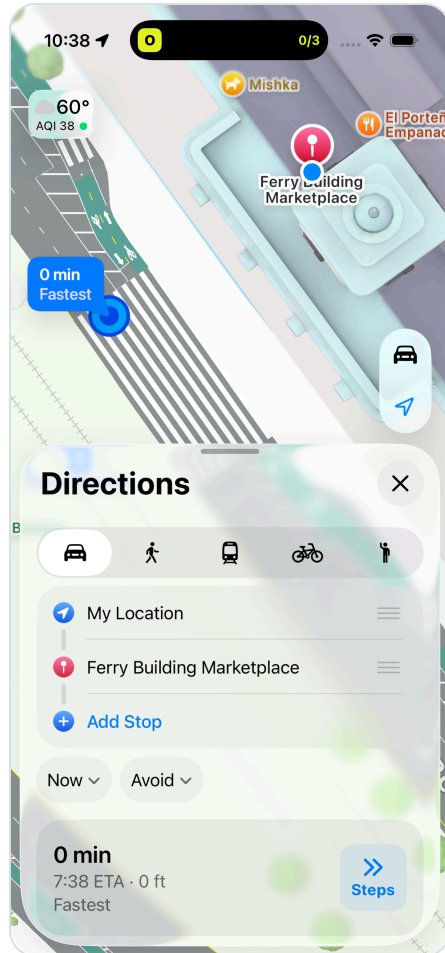
A short collection with a reason goes to a manager approval — the stop shows held, the banner says why, and the decision arrives by push while the driver keeps moving. Today's web version polls a screen every 20 seconds.

3 End of day

Collections tallied by method, sell-from-truck sales, truck count-back, and anything still in the approval queue — settled from local data, then GPS-stamped route check-out.

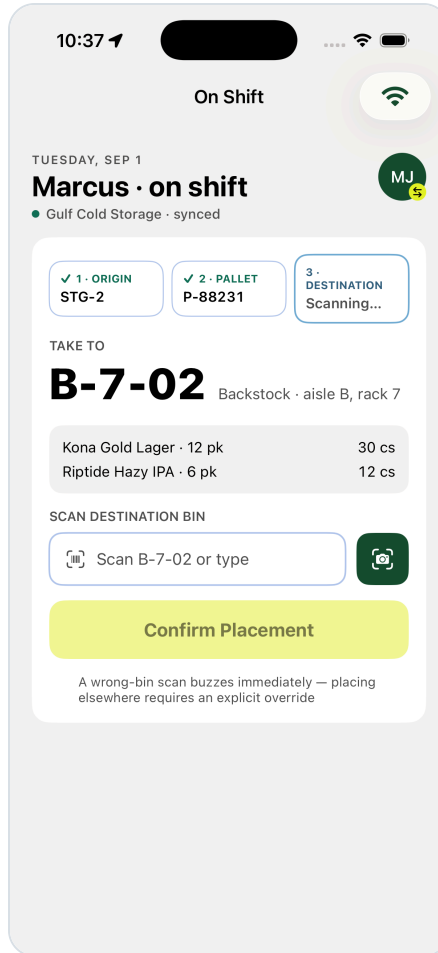
Native moments, up close

Three frames that only a native app can produce — the OS working for the front line, not just hosting a page.



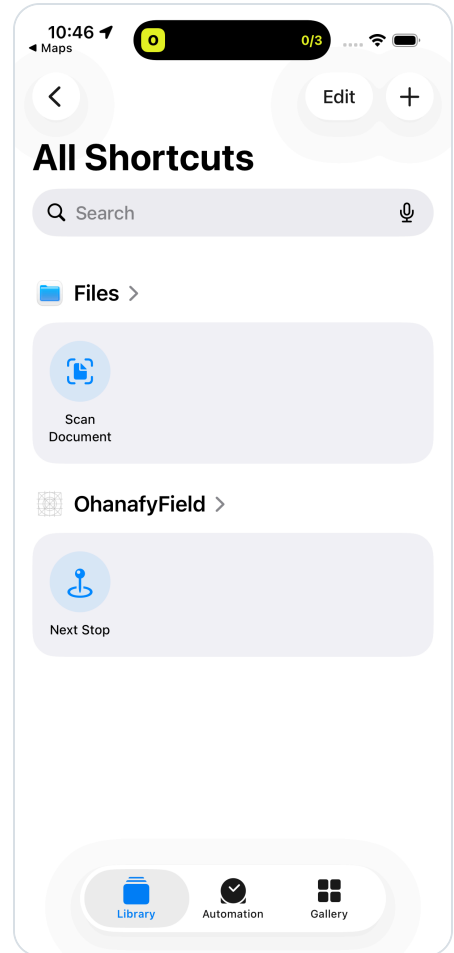
1 Two native features in one frame

Navigate hands the stop's coordinates to Apple Maps for turn-by-turn — and the Ohanafy route card stays live in the Dynamic Island the whole time. The shipped web version passes an address string and loses the user entirely.



2 Scan-first, glance-optional

The put-away's three scan legs — origin, pallet, destination — as one continuous flow with per-leg confirmation. On device this is the camera plus haptic and tone feedback; a keyboard-wedge scanner types into the same field.



3 Registered with the OS

The app's "Next Stop" intent in the system Shortcuts app — invocable by Siri voice on device, Spotlight, and automations, answering from on-device data. (Siri voice itself doesn't run in the Simulator; the registration shown here is what powers it.)

The native toolbox, named

Each capability in this document maps to a specific Apple framework already wired into the build — not a roadmap item.

FRAMEWORK	WHAT IT POWERS	STATUS
CoreLocation	Geofenced arrival detection (150 m region monitoring) and GPS-stamped check-ins	In this build
MapKit	The arrive screen's map with the geofence ring; coordinate handoff to Apple Maps directions	In this build
ActivityKit	The route Live Activity — lock-screen card and Dynamic Island, updated on every action	In this build
WidgetKit	Home-screen widget: route progress, next stop, sync state	In this build
App Intents	"What's my next stop?" via Siri, Shortcuts, and Spotlight — answered from on-device data	In this build
SwiftUI + SQLite	Instant screens over the on-device store; the durable command outbox under every journey	In this build
Sensory feedback (haptics)	Physical confirmation on scans, steppers, and check-in	In this build
AVFoundation	Continuous camera barcode scanning (the Simulator has no camera; scan gates simulate it)	Device build
Background URLSession · BGTaskScheduler · APNs	Photo uploads and sync that continue with the app closed; approval decisions arriving by push	Track A/C

The offline guarantees, as executable tests

The sync core beneath all three journeys is a standalone Swift library with an automated scenario suite. Each guarantee below is a passing test, not a promise.

GUARANTEE	PROVEN BY
Queued work survives app kills and device restarts	Outbox reopened from disk with the queue intact
Changes apply in the order the user acted	Ordered drain across mixed command types
A replayed command applies exactly once	Same command ID sent twice → one order server-side, same verdict returned
A rejection never strands the work behind it	Credit-hold refusal followed by an accepted payment in the same drain
An approval hold resolves the queue without losing the stop	Short-pay hold recorded and queryable; commands behind it still land
Offline and online are one code path	Unreachable drain leaves everything queued; reconnect drains with no user action
Salesforce stays the authority	Credit consumed across orders server-side; the second order refused

The suite runs as a standalone command (*swift run field-poc*) — 21 checks, all passing at the time of this document.

What this does that the Salesforce mobile app cannot

The Salesforce mobile app is a capable container for Lightning pages. These are the capabilities that sit outside what that container can reach — each one already demonstrated in this POC or standard iOS platform capability.

CAPABILITY	OHANAFY FIELD (NATIVE)	SALESFORCE MOBILE APP
Full workflows offline	Orders, surveys, payments complete and queue with zero signal; custom business flows included	Lightning pages need a connection; offline (Briefcase) can't run custom LWC experiences or Apex-backed logic
Instant screens	Every screen reads the on-device store — no spinners, no page loads	Server round-trips for search, pricing, and page loads
Lock screen & Dynamic Island	Live route card: progress, next stop, queued-changes count — zero app opens between stops	No Live Activities, no lock-screen presence
Geofenced arrival	Region monitoring detects the rep pulling in and readies check-in; GPS stamps automatically	No background location, no arrival detection
Home-screen widget	Route progress and next stop at a glance	Not available for custom experiences
Siri / Shortcuts	"What's my next stop?" answered from on-device data	Not available for custom experiences
Background transfer	Sync and photo uploads continue after the app is closed (background URLSession, silent push)	Work stops when the page closes
Haptic feedback	Physical confirmation on quantity changes and check-in — usable without looking	Not exposed to custom LWCs
One app, every role	Sales, warehouse, and driver journeys as modules in one binary; login decides the landing journey, a switcher covers multi-role users	Separate Lightning apps and pages navigated per role

Claims kept honest: barcode scanning and biometrics are deliberately absent from this table — the Salesforce mobile app does provide both to custom components. The differentiation case rests on offline depth, background execution, and OS surfaces, where the gap is structural.

Proven, pending, and the path forward

Proven in this POC

All three persona journeys over one shared offline core, with the avatar switcher standing in for login entitlements · durable offline queue with ordered, exactly-once replay · rejections and approval holds that never block the queue · offline promotion pricing · Live Activity, geofencing, widget, Siri, haptics · Ohanafy brand rendering natively.

Deliberately not yet built

Real Salesforce authentication and REST calls · delta sync of catalog and route data · photo upload pipeline · push notifications · multi-persona modules. Each has a designed seam in the architecture waiting for it.

Three parallel tracks from here

TRACK	WHAT	DONE MEANS
A — Wire to Salesforce	The first production API: response envelope, /v1/sales REST endpoints delegating to the shipped Sales Rep Home logic, OAuth login from the app, against a scratch org	A check-in tapped on the phone appears as a real record in the org — offline queue included
B — Deepen personas	The warehouse and driver journeys now run in the POC (this document); next is depth per the mockups — put-away and blind-count variants, camera barcode scanning on device, driver returns and credits	Each persona covers its full mockup scope on device hardware
C — Deeper native	Continue what this build started: background sync scheduling, push-driven updates to the lock-screen card, richer location features	The route card updates with the phone in a pocket

Tracks A and B/C run in parallel cleanly — the app talks to the server through a single interface, so the mock server and the real Salesforce transport are interchangeable without touching any screen.

Companion documents: **Front-Line Mobile Platform** (architecture, hybrid Salesforce + AWS design) and **Front-Line Journey Mockups** (three-persona design canvas), both available as shared review pages.